



LICENCIATURA EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA

Interação em Ambientes Virtuais

2º Semestre 2024/2025

Trabalho Prático 2

Docente: Engº Arnaldo Abrantes

11/05/2025

Turma: 62D

Trabalho realizado por:
Daniela Santos: A50032
Marta Ferreira: A50775

Conteúdo

Lista de Figuras	2
1 Introdução	3
2 O que são ML-Agents	3
3 Ambiente de Aprendizagem	4
3.1 Descrição do Cenário	4
3.2 Ambiente criado	4
4 Treino do agente	5
4.1 Algoritmo Utilizado	5
4.2 Análise das Observações	5
4.3 Sistema de Recompensas	6
5 Resultados Obtidos	6
6 Discussão dos Resultados	7
7 Conclusões	7
8 Bibliografia	8
Referências	8

Lista de Figuras

1	Funcionamento de ML-Agents	3
2	Ambiente criado em Unity	4
3	Gráfico de Recompensa Acumulada	6
4	Gráfico de Perda de Política	6

1 Introdução

Este segundo projeto da unidade curricular de Interação em Ambientes Virtuais tem como objetivo desenvolver um ambiente de aprendizagem no Unity utilizando a *toolbox* ML-Agents. A tarefa consiste em treinar um ou mais agentes para realizar uma ação específica com recurso a técnicas de aprendizagem por reforço.

O cenário pensado pelo grupo foca-se na precisão de manipulação de cor num determinado objeto com o uso de botões. O objetivo do agente é modificar a cor de um cilindro padrão através da interação com três botões (vermelho, verde e azul), de forma a igualar a cor de um cilindro de referência gerado aleatoriamente no início de cada episódio. Este cenário foi escolhido por permitir explorar de forma prática conceitos fundamentais da aprendizagem por reforço, tais como mapeamento de ações a estados, maximização de recompensas e exploração. Ao focar-se na manipulação precisa de variáveis contínuas (valores RGB), oferece um desafio simples à primeira vista, mas que exige decisões estratégicas por parte do agente.

2 O que são ML-Agents

ML-Agents é uma *toolbox open-source* desenvolvida pela Unity que permite treinar agentes inteligentes em ambientes 3D usando aprendizagem por reforço, aprendizagem por imitação e outros métodos.

A aprendizagem por reforço é uma abordagem em que um agente toma decisões com base nas observações do ambiente, ou seja, observam o estado atual, e as recompensas recebidas após cada ação. Ao longo do tempo, o agente ajusta o seu comportamento de forma a maximizar as recompensas acumuladas.

A arquitetura, retratada na Figura 1, envolve o Unity como simulador do ambiente e o Python como interface de treino, através do pacote *mlagents-learn*.

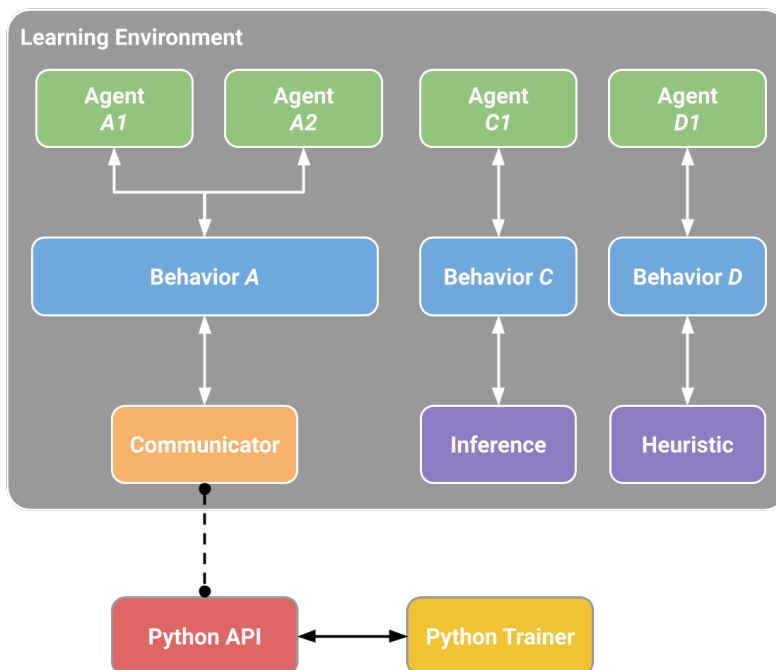


Figura 1: Funcionamento de ML-Agents

3 Ambiente de Aprendizagem

3.1 Descrição do Cenário

O ambiente consiste num plano limitado por paredes, contendo dois cilindros e três botões posicionados à frente do agente. Um dos cilindros assume uma cor aleatória em RGB no início de cada episódio, sendo este o “cilindro-alvo”. O segundo cilindro é o que o agente pode manipular, este vai interagir com três botões correspondentes às cores: Red, Green e Blue, podendo assim ajustar incrementalmente a cor do cilindro manipulável. O objetivo é igualar o máximo possível a cor deste cilindro ao cilindro-alvo.

3.2 Ambiente criado

Foi criado no Unity um cenário com os objetos mencionados anteriormente, como se pode ver na Figura 2. O agente é posicionado de frente para os três botões e para os dois cilindros (sendo o da esquerda o cilindro-alvo). A mecânica dos botões foi programada para alterar individualmente os canais R, G ou B da cor do cilindro manipulável.

Cada botão pode ser pressionado múltiplas vezes, e cada pressão incrementa o valor do respetivo canal de cor num valor fixo. Este tipo de controlo permite ao agente realizar ajustes finos e desenvolver uma estratégia mais precisa ao longo do treino.

É também possível visualizar no ecrã, através de UI, uma percentagem do quão próxima a cor está da desejada e quais valores RGB estão a ser alterados. A cor é considerada “correspondente” quando a diferença absoluta entre os valores RGB dos dois cilindros fica abaixo de um limiar pré-definido.

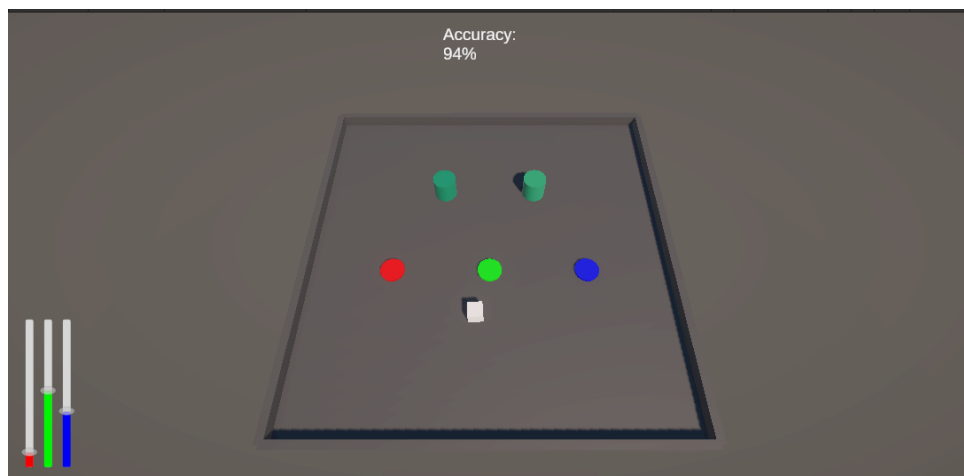


Figura 2: Ambiente criado em Unity

4 Treino do agente

4.1 Algoritmo Utilizado

Foi utilizado o algoritmo *Proximal Policy Optimization* (PPO), uma técnica de aprendizagem por reforço baseada em políticas, que tem como principal vantagem a estabilidade durante o treino. As políticas utilizadas pelo PPO têm pequenas restrições que evitam alterações bruscas no comportamento do agente e garantem uma aprendizagem mais estável, especialmente em ambientes complexos e contínuos como o deste projeto. Abaixo encontram-se os parâmetros mais relevantes utilizados no treino dos agentes, bem como uma breve explicação da sua função:

- **learning-rate:** $3.0e-4$ → um valor moderadamente alto, escolhido para acelerar o processo de aprendizagem inicial;
- **buffer-size:** 2048 → um valor mais elevado para permitir ao agente aprender a partir de uma amostra mais diversificada, o que melhora a generalização e suaviza o treino;
- **beta:** 0.03 → um valor mais elevado para que o agente explore mais o seu ambiente;
- **hidden-units:** 128 → foi usada uma rede com duas camadas de 128 neurónios cada, permitindo à rede aprender padrões mais complexos de cooperação entre os agentes;
- **gamma:** 0.99 → um valor alto para incentivar os agentes a pensarem a longo prazo.

Estes parâmetros foram cuidadosamente ajustados com o intuito de atingir um equilíbrio eficaz entre a rapidez de aprendizagem e a estabilidade do comportamento do agente durante o treino. Valores como o **learning-rate** mais elevado e o **buffer-size** relativamente grande permitem acelerar o processo de aprendizagem, assegurando que o agente consegue adquirir comportamentos úteis em menos episódios. Esta abordagem foi essencial num cenário em que o agente precisa de realizar múltiplas ações sequenciais com precisão. Assim, o conjunto de hiperparâmetros escolhidos reflete um compromisso entre explorar rapidamente estratégias eficazes e evitar comportamentos erráticos ou inconsistentes.

4.2 Análise das Observações

Para que um agente possa aprender a agir corretamente num ambiente virtual, é essencial que tenha acesso a informações relevantes sobre o estado desse ambiente. Neste cenário, o agente pode observar os seguintes elementos:

- **Coordenadas do Agente:** permitem ao agente conhecer a sua posição relativa no ambiente, o que é útil para orientar-se em relação aos botões e objetos à sua volta;
- **Coordenadas de cada Botão:** necessárias para que o agente saiba onde se encontram os botões com que pode interagir;
- **Cor do cilindro-alvo:** representa o objetivo da tarefa, o agente precisa saber quais valores RGB deve tentar replicar;
- **Cor do cilindro manipulável:** indica o estado atual do cilindro que o agente pode alterar;

Com estas observações, o agente tem informação suficiente para decidir qual botão pressionar em cada passo para aproximar os valores de cor.

4.3 Sistema de Recompensas

O sistema de recompensas foi feito de modo a incentivar o agente a alcançar o objetivo de igualar a cor do cilindro manipulável à cor do cilindro-alvo de forma eficiente e precisa. Foram definidas diferentes penalizações e recompensas de acordo com o comportamento do agente:

- O agente recebe uma **penalização de -1.5** sempre que colide com as paredes do ambiente, desencorajando movimentos aleatórios ou descontrolados que o afastem do objetivo;
- A recompensa principal é calculada de forma linear inversa em função do erro da cor, ou seja, quanto menor a diferença entre os valores RGB dos dois cilindros, maior a **recompensa (variando entre 0 e 1)**. Este mecanismo guia o agente progressivamente para uma aproximação mais precisa à cor correta;
- Sempre que o erro da cor for superior a 0.8, o agente recebe uma **penalização adicional de -0.2 por ação**, o que força o agente a evitar permanecer em estados muito distantes da solução, incentivando-o a explorar ações mais eficazes;
- Quando a diferença entre as cores é inferior a 0.05, considera-se que as cores estão suficientemente próximas, e o agente recebe a **recompensa máxima de 1**, indicando o sucesso no episódio;

5 Resultados Obtidos

O primeiro gráfico da Figura 3 reflete o desempenho global do agente durante a aprendizagem. Ele mostra como a recompensa acumulada aumenta ao longo do tempo, o que é essencial para verificar se o agente está realmente a aprender e se o desempenho está a melhorar. É possível observar uma tendência crescente no gráfico o que indica progresso.

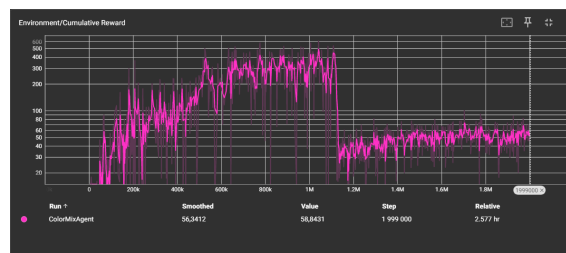


Figura 3: Gráfico de Recompensa Acumulada

O gráfico da Figura 4 é essencial para entender se o agente está a aprender uma boa política, se a perda de política diminui ao longo do tempo, significa que o agente está a melhorar a sua capacidade de tomar decisões de forma mais eficiente. Neste gráfico podemos observar que ele apresenta-se relativamente constante ao longo do treino, isto indica que o agente estabilizou numa determinada política e deixou de apresentar melhorias significativas, mantendo-se consistente.

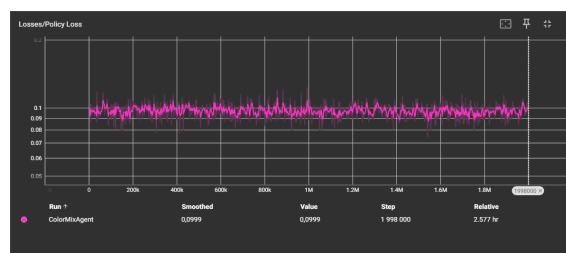


Figura 4: Gráfico de Perda de Política

6 Discussão dos Resultados

Os resultados obtidos demonstram que o projeto alcançou com sucesso os objetivos principais. O agente treinado foi capaz de interagir com o ambiente e aprender a realizar a tarefa proposta: igualar a cor de um cilindro ao de referência, utilizando apenas recompensas e observações do estado atual.

O uso do algoritmo PPO foi adequado, garantindo estabilidade ao longo do treino e evitando comportamentos erráticos. A escolha dos hiperparâmetros contribuiu para uma aprendizagem eficaz, permitindo ao agente desenvolver uma política que mostrou-se funcional e capaz de atingir bons resultados na maioria dos episódios.

Contudo, como é possível ver na Figura 4, nem tudo foi perfeito. O comportamento constante da **policy loss** levanta a hipótese de estagnação no processo de aprendizagem, indicando que ainda há espaço para melhorar a capacidade de generalização do agente.

7 Conclusões

O desenvolvimento deste projeto permitiu uma compreensão prática e aprofundada dos conceitos de interação em ambientes virtuais e de aprendizagem por reforço, aplicados num contexto realista através da ferramenta ML-Agents da Unity.

O agente foi capaz de aprender a tarefa proposta com um grau de sucesso considerável, adaptando-se ao ambiente e maximizando as recompensas com base nas observações fornecidas. A estrutura de recompensas e penalizações mostrou-se adequada para guiar o processo de aprendizagem, incentivando estratégias mais precisas e eficientes ao longo do tempo.

Apesar dos resultados positivos, a análise da *policy loss* indicou uma possível estagnação na aprendizagem, o que abre espaço para futuras melhorias, como o ajuste de hiperparâmetros, alteração do sistema de recompensas ou a introdução de maior variabilidade no ambiente.

Em suma, este trabalho não só consolidou os conhecimentos adquiridos ao longo da unidade curricular, como também despertou o interesse para investigações futuras no domínio da inteligência artificial aplicada à simulação interativa e ao treino de agentes autónomos.

8 Bibliografia

Referências

- [1] Unity-Technologies, *Unity ML Agents Toolkit*
<https://github.com/Unity-Technologies/ml-agents>,
2025.
- [2] Abrantes, A, *Videos de Tutorial de ML Agents no Moodle da cadeira*,
2025.